



Aula 8 – Página de Erro / Preloading / SuspenseAPI / Error Boundary

Objetivos

- Criar uma página de **erro 404** customizada
- Implementar **carregamento (loading)** ao buscar ou renderizar dados
- Usar `React.Suspense` para carregamento de páginas com **lazy loading**
- Criar um **Error Boundary** para capturar e exibir erros de forma amigável

Página de Erro (404)

`src/pages/NotFound.tsx`

```
import { Link } from 'react-router-dom'; export function NotFound() { r
eturn ( <div className="text-center py-20"> <h1 className="text-4xl fon
t-bold mb-4">404 - Página não encontrada</h1> <p className="text-gray-6
00 mb-6">Ops! Parece que você se perdeu.</p> <Link to="/" className="te
xt-blue-600 hover:underline"> Voltar para a Home </Link> </div> ); }
```

Adicionando a rota 404

src/App.tsx (atualização)

```
import { BrowserRouter, Routes, Route } from 'react-router-dom'; import
{ Layout } from './components/Layout'; import { Home } from './pages/Ho
me'; import { AddWorkout } from './pages/AddWorkout'; import { WorkoutD
etails } from './pages/WorkoutDetails'; import { NotFound } from './pag
es/NotFound'; function App() { return ( <BrowserRouter> <Routes> <Route
path="/" element={<Layout />} > <Route index element={<Home />} /> <Rout
e path="add" element={<AddWorkout />} /> <Route path="workout/:id" elem
ent={<WorkoutDetails />} /> <Route path="*" element={<NotFound />} />
</Route> </Routes> </BrowserRouter> ); } export default App;
```

Preloading (estado de carregamento)

Crie um componente visual simples para indicar carregamento.

src/components>Loading.tsx

```
export function Loading() { return ( <div className="text-center py-1
0"> <p className="animate-pulse text-gray-500">Carregando...</p> </div>
); }
```

Lazy Loading com SuspenseAPI

src/App.tsx com lazy

```
import { BrowserRouter, Routes, Route } from 'react-router-dom'; import
{ Layout } from './components/Layout'; import { Loading } from './compo
nents>Loading'; import { Suspense, lazy } from 'react'; const Home = la
zy(() => import('./pages/Home').then(m => ({ default: m.Home }))); cons
t AddWorkout = lazy(() => import('./pages/AddWorkout').then(m => ({ def
ault: m.AddWorkout }))); const WorkoutDetails = lazy(() => import('./pa
ges/WorkoutDetails').then(m => ({ default: m.WorkoutDetails }))); const
NotFound = lazy(() => import('./pages/NotFound').then(m => ({ default:
m.NotFound }))); function App() { return ( <BrowserRouter> <Suspense fa
llback={<Loading />} > <Routes> <Route path="/" element={<Layout />} > <R
oute index element={<Home />} /> <Route path="add" element={<AddWorkout
/>} /> <Route path="workout/:id" element={<WorkoutDetails />} /> <Route
path="*" element={<NotFound />} /> </Route> </Routes> </Suspense> </Bro
wserRouter> ); } export default App;
```

Error Boundary

- Usado para capturar erros de **renderização** no React.
- Não captura erros de eventos ou código assíncrono.

`src/components/ErrorBoundary.tsx`

```
import { Component, ReactNode } from 'react'; interface Props { children: ReactNode; } interface State { hasError: boolean; } export class ErrorBoundary extends Component<Props, State> { constructor(props: Props) { super(props); this.state = { hasError: false }; } static getDerivedStateFromError() { return { hasError: true }; } componentDidCatch(error: any, info: any) { console.error("Erro capturado pelo ErrorBoundary:", error, info); } render() { if (this.state.hasError) { return ( <div className="text-center py-20"> <h1 className="text-2xl font-bold mb-4">Algo deu errado 🤔</h1> <p>Tente atualizar a página ou volte mais tarde.</p> </div> ); } return this.props.children; } }
```

Checklist da Aula

- ☐ Criou página de erro 404
- ☐ Adicionou rota coringa
- ☐ Implementou componente de loading
- ☐ Aplicou `React.Suspense` para lazy loading
- ☐ Criou um Error Boundary para capturar erros

Atividade prática

1. Personalizar a página 404 com imagem ou ícone.
2. Adicionar animação CSS no loading.
3. Simular um erro proposital para testar o `ErrorBoundary`.

